Table of contents

Series 5: Program Verification with Why3

Methods

Formal Specification with Preconditions and Postconditions

Description: In program verification, expected behavior is specified using logical assertions.

- **Precondition**: Formula that must be true before program execution (assumptions about inputs).
- **Postcondition**: Formula that must be true after execution (guarantees about outputs). These conditions are expressed in first-order logic, often with quantifiers over array indices and values.

Loop Invariants

Description: A **loop invariant** is a logical formula that remains true before and after each iteration of a loop. It allows reasoning about partial correctness and termination. For a `while` loop, the invariant must be:

- true before the first iteration,
- preserved by the loop body (if the invariant is true before the iteration and the guard condition is true, then it remains true after the iteration),
- strong enough to imply the postcondition when the loop condition becomes false.

Auxiliary Predicates

Description: To simplify specifications, **predicates** (boolean functions) are defined to encapsulate reusable properties (e.g., permutation, sorted). These predicates can be defined in Why3 and used in invariants, preconditions, and postconditions.

Array Properties

Description: Algorithms on arrays require expressing global properties:

- **Sorted**: increasing or decreasing order of elements.
- **Permutation**: preservation of the multiset of elements (same occurrences, possibly different order).
- **Sub-array**: properties over index segments.

Partial Correctness Proof (Hoare Logic)

Description: **Hoare logic** allows decomposing the proof of program correctness by verifying triples `{P} C {Q}`. For loops, an invariant is used. Why3 automates the generation of proof obligations (verification conditions) from annotated code.

Reminder

- **De Morgan's Laws**: $\neg(A \wedge B) \equiv \neg A \vee \neg B$ $\neg(A \vee B) \equiv \neg A \wedge \neg B$
- **Implication Elimination**: $A \Rightarrow B \equiv \neg A \vee B$
- **Distributivity**: $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$ $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
- **Quantifiers**: $\forall i, P(i)$: for all i, $P(i)$ is true. $\exists i, P(i)$: there exists i such that $P(i)$ is true. $\neg\forall i, P(i) \equiv \exists i, \neg P(i)$ $\neg\exists i, P(i) \equiv \forall i, \neg P(i)$
- **Hoare Triple**: $\{P\}\ C\ \{Q\}$ means: if execution of C begins in a state satisfying P, and if C terminates, then the final state satisfies Q.

Illustrative Examples

Example 1: Formal Specification with Preconditions and Postconditions

Context: Binary search in a sorted array.

Application:

The pseudocode for binary search is given. The **precondition** requires that the input array be sorted:

$$\forall i, j \in \{0, \ldots, n-1\},\ i \leq j \Rightarrow a[i] \leq a[j]$$

The **postcondition** specifies the result `res`:

$$(res \in \{0, \ldots, n-1\} \wedge a[res] = elem)\ \vee\ (res = -1 \wedge \forall i \in \{0, \ldots, n-1\}, a[i] \neq elem)$$

This guarantees that if the element is present, `res` is a valid index where it is found; otherwise `res` is -1 and the element is absent.

Example 2: Loop Invariants

Context: Binary search, `while` loop.

Application:

The proposed invariant is:

$$I_1\ =\ 0 \leq l\ \wedge\ u \leq n-1\ \wedge\ \forall i \in \{0, \ldots, n-1\},\ a[i] = elem \Rightarrow l \leq i \leq u$$

where `l` and `u` are the current bounds of the search interval.

Interpretation:

- The bounds `l` and `u` remain within the array limits (or at the edges).
- If the element `elem` is present in the array, then it must be within the interval `[l, u]`.

At each iteration, the middle `m` is computed. Depending on the comparison between `a[m]` and `elem`, `l` or `u` is updated to preserve the invariant. When the loop terminates (`l > u`), the postcondition can be concluded.

Example 3: Array Properties and Auxiliary Predicates

Context: Bubble sort.

Application:

The **postcondition** must express that the output array is sorted and is a permutation of the input array. An auxiliary predicate `permut_all` is defined (for example):

```
predicate permut_all (a1: array int) (a2: array int) =
  length a1 = length a2 /\
  forall v: int. occurrences v a1 = occurrences v a2
```

The postcondition is then written as:

$$\text{sorted}(a) \wedge \text{permut\_all}(a, a_{\text{initial}})$$

where `sorted` is a predicate expressing increasing order.

For **nested loop invariants**:

- **Outer loop** (`i`): the elements of `a[i+1 .. n-1]` are sorted and are the largest; the subarray `a[0 .. i]` is a permutation of the corresponding initial subarray.
- **Inner loop** (`j`): the subarray `a[0 .. j-1]` is sorted and all its elements are less than or equal to `a[j]` (for ascending version).

These invariants allow proving that each pass places the largest remaining element in its final position.

Synthesis

This series of exercises illustrates fundamental methods of formal verification of imperative programs with Why3. Key points are:

1. **Rigorous specification** via preconditions and postconditions in first-order logic.
2. **Use of loop invariants** to capture intermediate state and guide the proof.
3. **Definition of auxiliary predicates** to abstract complex properties (sorted, permutation).
4. **Application to classic algorithms**: binary search, bubble sort, selection sort.

These methods allow demonstrating **partial correctness** (if the program terminates, it satisfies its specification) and, with additional arguments (variants), **termination**.